嵌入式 Linux 应用开发完全手册之文件 IO

深圳百问网科技有限公司

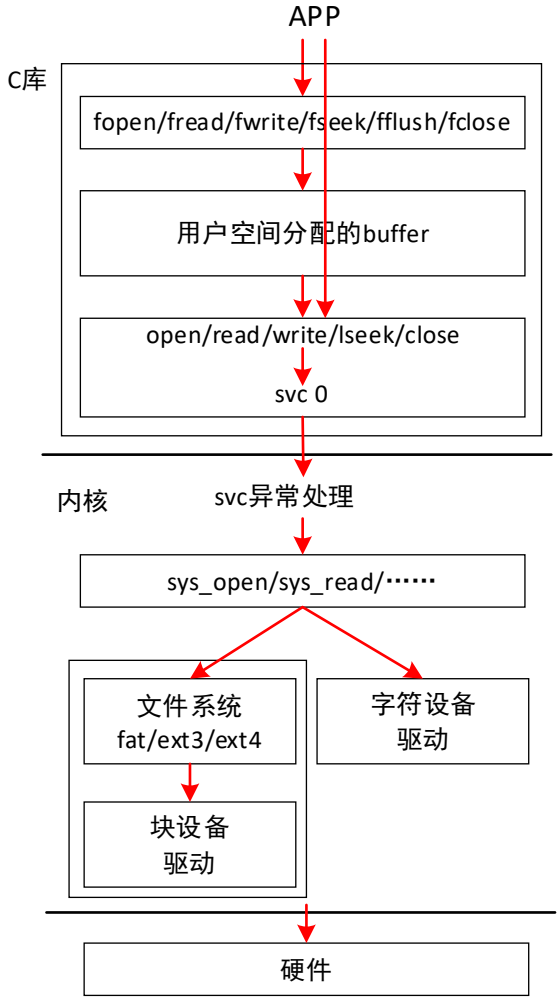
2023.8.22

第1章 文件基础操作

1.1 文件 IO 函数分类

在 Linux 上操作文件时，有两套函数：标准 IO、系统调用 IO。标准 IO 的相关函数是：fopen/fread/fwrite/fseek/fflush/fclose 等。系统调用 IO 的相关函数是：open/read/write/lseek/fsync/close。

这 2 种 IO 函数的差别如下图所示：



标准 IO 的内部，会分配一个用户空间的 buffer，读写操作先经过这个 buffer。在有必要时，才会调用底下的系统调用 IO 向内核发起操作。

所以：标准 IO 效率更高；但是要访问驱动程序时就不能使用标准 IO，而是使用系统调用 IO。

1.2 函数原型

1.2.1 系统调用接口

下表列出了 open/read/write/lseek/fsync/close 这几个函数的用法。

函数名	函数原型	描述

open	#include <sys/types.h>
	#include <sys/stat.h>
	#include <fcntl.h>
	int open(const char *pathname, int flags);
	int open(const char *pathname, int flags, mode_t mode);
		作用：打开或者创建一个文件
		pathname：文件路径名，或者文件名
		flags：表示打开文件所采用的操作
		O_RDONLY：只读模式
		O_WRONLY：只写模式
		O_RDWR：可读可写
		O_APPEND 表示追加，如果原来文件里面有内容，则这次写入会写在文件的最末尾。
		O_CREAT 表示如果指定文件不存在，则创建这个文件
		O_EXCL 表示如果要创建的文件已存在，则出错，同时返回 -1，并且修改 errno 的值。
		O_TRUNC 表示截断，如果文件存在，并且以只写、读写方式打开，则将其长度截断为 0。
		O_NOCTTY 如果路径名指向终端设备，不要把这个设备用作控制终端。
		O_NONBLOCK 如果路径名指向 FIFO/块文件/字符文件，则把文件的打开和后继 I/O 设置为非阻塞模式（nonblocking mode）
		O_DSYNC 等待物理 I/O 结束后再 write。在不影响读取新写入的数据的前提下，不等待文件属性更新。
		O_RSYNC read 等待所有写入同一区域的写操作完成后再进行
		O_SYNC 等待物理 I/O 结束后再 write，包括更新文件属性的 I/O
		mode：文件访问权限的初始值

read	#include <unistd.h> ssize_t read(int fd, void *buf, size_t count);	**作用**：从给定的文件描述符指定的文件中，读取 count 个字节的数据，存放至 buf 中。 **fd**：指定要读写的文件描述符 **buf**：缓冲区，一般是一个数组，用于存放读取的内容 **count**：一次要读取的最大字节数
write	#include <unistd.h> ssize_t write(int fd, const void *buf, size_t count);	**作用**：将 buf 中的 count 字节数据写入指定文件描述符的文件中。 **fd**：指定要写入的文件描述符 **buf**：缓冲区，一般是一个数组，读取存放于该数组的内容存放于文件中 **count**：要写入的实际字节数

dup
dup2
dup3
	#include <unistd.h>
	int dup(int oldfd);
	int dup2(int oldfd, int newfd);
	#define _GNU_SOURCE
	#include <fcntl.h>
	#include <unistd.h>
	int dup3(int oldfd, int newfd, int flags);
		作用：复制文件描述符，就是两个文件句柄指向同一个文件描述符，这两个文件句柄共享文件偏移地址、状态
		oldfd：被复制的文件句柄
		newfd：复制得到的文件句柄
		注意：dup 函数返回的文件句柄是“未使用的最小文件句柄”，
		dup2 可以指定复制得到的文件句柄为 newfd，
		dup3 跟 dup2 类似，flags 参数必要么是 0，要么是 O_CLOEXEC

lseek	#include <sys/types.h> #include <unistd.h> off_t lseek(int fd, off_t offset, int whence);	**作用**：重新定位读/写文件偏移 **fd**：指定要偏移的文件描述符 **offset**：文件偏移量 **whence**：开始添加偏移 offset 的位置 SEEK_SET，offset 相对于文件开头进行偏移 SEEK_CUR，offset 相对文件当前位置进行偏移 SEEK_END，offset 相对于文件末尾进行偏移
fsync	#include <unistd.h> int fsync(int fd);	**作用**：同步内存中所有已修改的文件数据到储存设备 **fd**：指定要同步的文件描述符

close	#include <sys/types.h> #include <sys/stat.h> #include <fcntl.h> int close(int fd);	**作用**：关闭已打开的文件

1.2.2 标准 IO 接口

列出 fopen/fread/fwrite/fseek/fflush/fclose 这几个函数的用法。

函数名	函数原型	描述
fopen	#include<stdio.h> FILE *fopen(const char *filename, const char *mode);	**作用**：打开或者创建一个文件 **filename**：文件路径名，或者文件名 **mode**：文件的访问模式 r 打开一个用于读取的文件。该文件必须存在。 w 创建一个用于写入的空文件。如果文件名称与已存在的文件相同，则会删除已有文件的内容，文件被视为一个新的空文件。 a 追加到一个文件。写操作向文件末尾追加数据。如果文件不存在，则创建文件。 r+ 打开一个用于更新的文件，可读取也可写入。该文件必须存在。 w+ 创建一个用于读写的空文件。 a+ 打开一个用于读取和追加的文件。
fread	#include<stdio.h> size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);	**作用**：从给定流 stream 读取数据到 ptr 所指向的数组中 **ptr**：指向带有最小尺寸 size*nmemb 字节的内存块的指针 **size**：这是要读取的每个元素的大小，以字节为单位 **nmemb**：元素的个数，每个元素的大小为 size 字节 **stream**：这是指向 FILE 对象的指针，该 FILE 对象指定了一个输入流
fwrite	#include <stdio.h> size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);	**作用**：把 ptr 所指向的数组中的数据写入到给定流 stream 中 **ptr**：这是指向要被写入的元素数组的指针 **size**：这是要被写入的每个元素的大小，以字节为单位 **nmemb**：这是元素的个数，每个元素的大小为 size 字节 **stream**：这是指向 FILE 对象的指针，该 FILE 对象指定了一个输出流
fseek	#include <stdio.h> int fseek(FILE *stream, long int offset, int whence);	**作用**：设置流 stream 的文件位置为给定的偏移 offset **stream**：这是指向 FILE 对象的指针，该 FILE 对象标识了流 **offset**：这是相对 whence 的偏移量，以字节为单位 **whence**：表示开始添加偏移 offset 的位置 SEEK_SET，offset 相对于文件开头进行偏移 SEEK_CUR，offset 相对文件当前位置进行偏移 SEEK_END，offset 相对于文件末尾进行偏移
fflush	#include <stdio.h> int fflush(FILE *stream);	**作用**：刷新流 stream 的输出缓冲区 **stream**：这是指向 FILE 对象的指针，该 FILE 对象指定了一个缓冲流
fclose	#include <stdio.h> int fclose(FILE *stream);	**作用**：关闭流 stream,且刷新其缓冲区 **stream**：这是指向 FILE 对象的指针，该 FILE 对象指定了要被关闭的流

1.3 现场编程

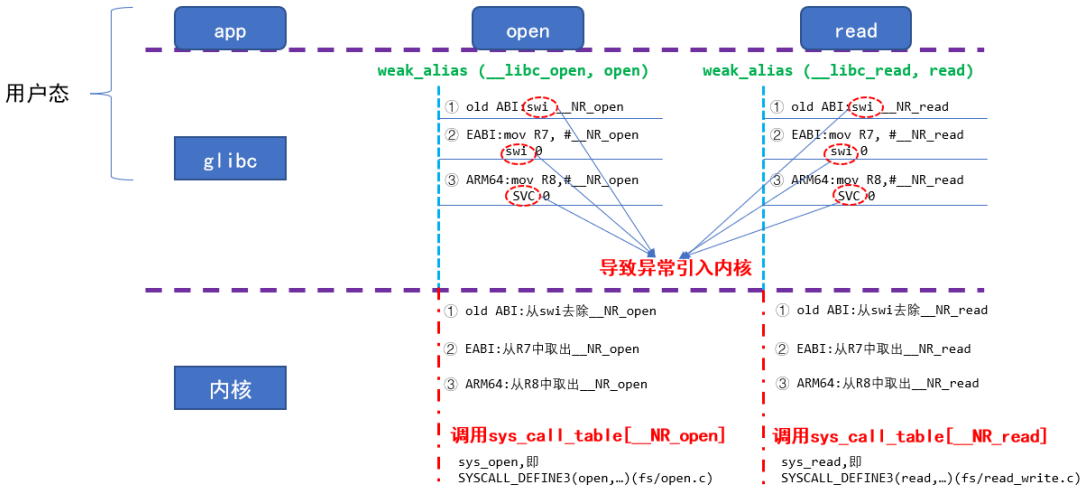
1.3.1 系统调用接口

1.3.2 标准 IO 接口

1.4 内部机制

1.4.1 系统调用接口内部流程

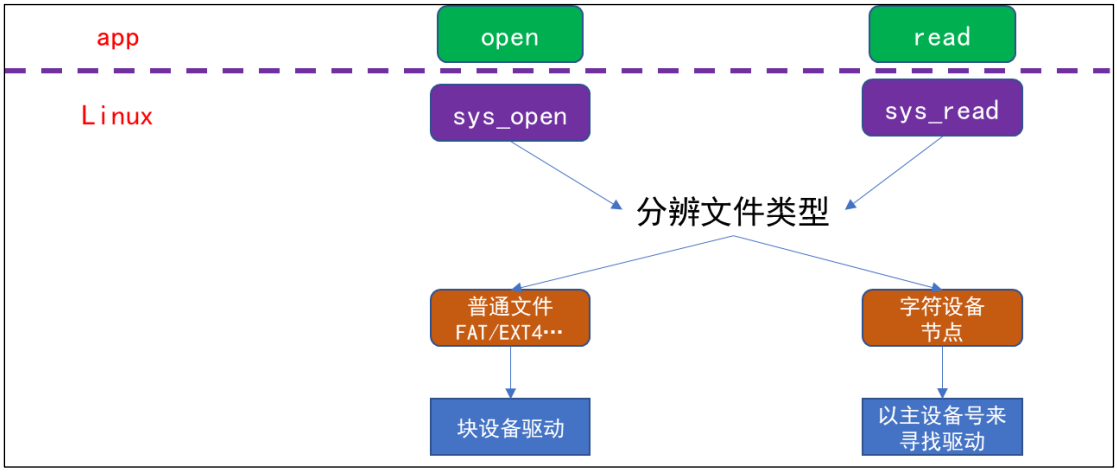
系统调用的内部流程：



以 open/read 为例，从用户态调用 API 触发异常进入内核的过程如上图所示，最后调用的 `sys_call_table` 的函数指针数组如下：

```
/* 0 */ CALL(sys_restart_syscall)
        CALL(sys_exit)
        CALL(sys_fork)
        CALL(sys_read)
        CALL(sys_write)
/* 5 */ CALL(sys_open)
        CALL(sys_close)
        CALL(sys_ni_syscall)
        CALL(sys_creat)
        CALL(sys_link)
/* 10 */CALL(sys_unlink)
        CALL(sys_execve)
        CALL(sys_chdir)
        CALL(OBSULETE(sys_time))
```

1.4.2 内核的 sys_open、sys_read 会做什么？



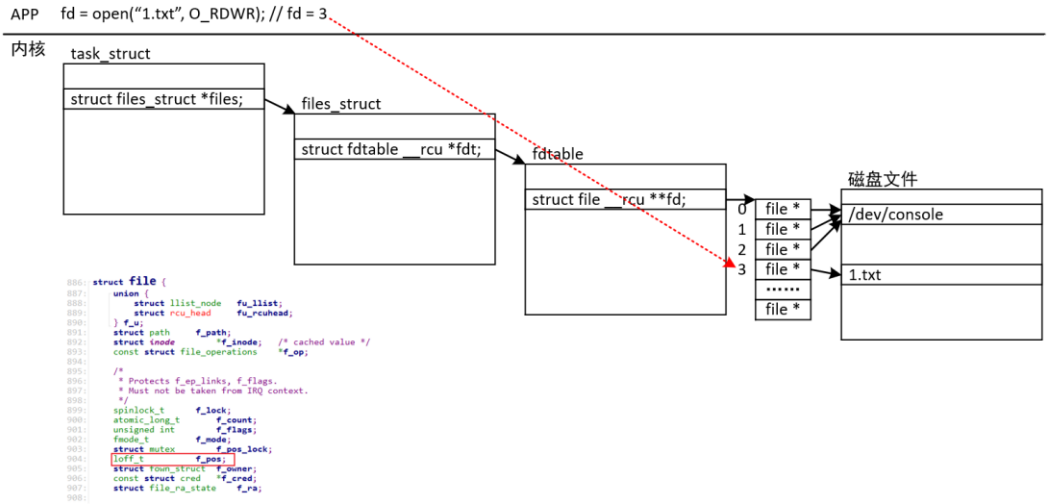
进入内核后，sys_read/open 会首先判断文件的类型，进而通过文件系统的代码读写普通文件，或者通过设备驱动程序访问硬件。

对于 sys_open，它的实质如下：

```
1072: SYSCALL_DEFINE3(open, const char __user *, filename, int, flags, umode_t, mode)
1073: {
1074:     if (force_o_largefile())
1075:         flags |= O_LARGEFILE;
1076:
1077:     return do_sys_open(AT_FDCWD, filename, flags, mode);
1078: }

1044: long do_sys_open(int dfd, const char __user *filename, int flags, umode_t mode)
1045: {
1046:     struct open_flags op;
1047:     int fd = build_open_flags(flags, mode, &op);
1048:     struct filename *tmp;
1049:
1050:     if (fd)
1051:         return fd;
1052:
1053:     tmp = getname(filename);
1054:     if (IS_ERR(tmp))
1055:         return PTR_ERR(tmp);
1056:
1057:     fd = get_unused_fd_flags(flags);   1.获得未用的文件句柄
1058:     if (fd >= 0) {
1059:         struct file *f = do_filp_open(dfd, tmp, &op);   2.打开文件，得到file结构体
1060:         if (IS_ERR(f)) {
1061:             put_unused_fd(fd);
1062:             fd = PTR_ERR(f);
1063:         } else {
1064:             fsnotify_open(f);
1065:             fd_install(fd, f);   3.在当前进程里记录下来
1066:         }
1067:     }
1068:     putname(tmp);
1069:     return fd;
1070: } « end do_sys_open »
```

先获得未使用的文件句柄，然后打开文件得到一个 file 结构体，最后在当前进程记录下这个文件：记录“文件句柄、file 结构体”的对应关系。以后，APP 使用文件句柄操作文件时，在内核里面就是根据对应的 file 结构体去操作文件。open 文件后遗留的结果如下：



file 结构体内部结构如下：

```
886: struct file {
887:     union {
888:         struct llist_node   fu_llist;
889:         struct rcu_head     fu_rcuhead;
890:     } f_u;
891:     struct path     f_path;
892:     struct inode        *f_inode;   /* cached value */
893:     const struct file_operations    *f_op;
894:
895:     /*
896:      * Protects f_ep_links, f_flags.
897:      * Must not be taken from IRQ context.
898:      */
899:     spinlock_t      f_lock;
900:     atomic_long_t       f_count;
901:     unsigned int        f_flags;
902:     fmode_t         f_mode;
903:     struct mutex        f_pos_lock;
904:     loff_t          f_pos;
905:     struct fown_struct  f_owner;
906:     const struct cred   *f_cred;
907:     struct file_ra_state    f_ra;
908:
909:     u64         f_version;
910: #ifdef CONFIG_SECURITY
911:     void            *f_security;
912: #endif
913:     /* needed for tty driver, and maybe others */
914:     void            *private_data;
915:
916: #ifdef CONFIG_EPOLL
917:     /* Used by fs/eventpoll.c to link all the hooks to this file */
918:     struct list_head    f_ep_links;
919:     struct list_head    f_tfile_llink;
920: #endif /* #ifdef CONFIG_EPOLL */
921:     struct address_space    *f_mapping;
922: } « end file »  __attribute__((aligned(4)));     /* lest something weird decides that 2 is OK */
923:
```

里面记录了很多信息，比如“当前读写位置”f_pos：当我们执行 lseek、read、write 函数时，内核都会更新 file 结构体的 f_pos。